

A2 Connecting to the database

Core Concepts

1. **Persistence & Disk Storage:** Unlike storing data in a Python list (which wipes on application restart), SQLite saves records into a single file on disk (tasks.db). The data outlives the server process.
2. **Parameterized Queries (? Placeholders):** Values are passed separately from the SQL string (e.g., WHERE id = ?, (task_id,)). This prevents SQL Injection attacks by ensuring user input is treated strictly as data, never executable code.
3. **Idempotent Database Initialization:** CREATE TABLE IF NOT EXISTS ensures schema setup runs smoothly on every launch without raising errors. Seeding defaults only when COUNT(*) == 0 prevents duplicate records on every restart.
4. **Row Factories (sqlite3.Row):**

Configuring `conn.row_factory = sqlite3.Row` allows accessing query result columns by key name (e.g., `row["title"]`) instead of raw tuple indices (`row[1]`).

Implementation (main.py)

Python

```
import sqlite3
from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel

DB_NAME = "tasks.db"

# --- DATABASE HELPERS ---

def get_db_connection():
    """Establishes connection to tasks.db and configures dict-like row access."""
    conn = sqlite3.connect(DB_NAME)
    conn.row_factory = sqlite3.Row
    return conn

def init_db():
    """
    Stage 0: Creates the tasks table if missing and seeds initial records
    only when the table is empty to avoid duplicating data on restarts.
```

```

"""
conn = get_db_connection()
cursor = conn.cursor()

# Create table with id (primary key), title (text), and done (0/1 boolean)
cursor.execute("""
    CREATE TABLE IF NOT EXISTS tasks (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT NOT NULL,
        done INTEGER NOT NULL DEFAULT 0
    );
""")

# Seed initial 3 tasks only if empty
cursor.execute("SELECT COUNT(*) FROM tasks;")
if cursor.fetchone()[0] == 0:
    initial_tasks = [
        ("Buy groceries", 0),
        ("Read documentation", 0),
        ("Complete Week 3 Assignment", 0),
    ]
    cursor.executemany(
        "INSERT INTO tasks (title, done) VALUES (?, ?);", initial_tasks
    )
    conn.commit()

conn.close()

```

--- APP INITIALIZATION ---

```
app = FastAPI()
```

```

@app.on_event("startup")
def startup_event():
    init_db()

```

--- PYDANTIC SCHEMAS ---

```

class TaskCreate(BaseModel):
    title: str

```

```

class TaskUpdate(BaseModel):
    title: str
    done: bool

```

```
# --- STAGE 1: READ ENDPOINTS ---
```

```
@app.get("/tasks")
def get_all_tasks():
    """Fetches all tasks from the SQLite database."""
    conn = get_db_connection()
    tasks = conn.execute("SELECT id, title, done FROM tasks;").fetchall()
    conn.close()

    return [
        {"id": t["id"], "title": t["title"], "done": bool(t["done"])}
        for t in tasks
    ]

@app.get("/tasks/{task_id}")
def get_task(task_id: int):
    """Fetches a single task by ID using parameterized SQL."""
    conn = get_db_connection()
    task = conn.execute(
        "SELECT id, title, done FROM tasks WHERE id = ?;", (task_id,)
    ).fetchone()
    conn.close()

    if not task:
        raise HTTPException(
            status_code=404, detail={"error": "Task not found"}
        )

    return {"id": task["id"], "title": task["title"], "done": bool(task["done"])}
```

```
# --- STAGE 2: CREATE ENDPOINT ---
```

```
@app.post("/tasks", status_code=status.HTTP_201_CREATED)
def create_task(task: TaskCreate):
    """Inserts a new task and returns the newly generated auto-incremented ID."""
    if not task.title.strip():
        raise HTTPException(
            status_code=400, detail={"error": "Title cannot be empty"}
        )

    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute(
```

```

        "INSERT INTO tasks (title, done) VALUES (?, ?);", (task.title, 0)
    )
    new_id = cursor.lastrowid
    conn.commit()
    conn.close()

    return {"id": new_id, "title": task.title, "done": False}

```

--- STAGE 3: UPDATE & DELETE ENDPOINTS ---

```

@app.put("/tasks/{task_id}")
def update_task(task_id: int, task: TaskUpdate):
    """Updates title and done status for an existing task."""
    if not task.title.strip():
        raise HTTPException(
            status_code=400, detail={"error": "Title cannot be empty"}
        )

    conn = get_db_connection()
    cursor = conn.cursor()

    cursor.execute(
        "UPDATE tasks SET title = ?, done = ? WHERE id = ?;",
        (task.title, int(task.done), task_id),
    )
    conn.commit()

    if cursor.rowcount == 0:
        conn.close()
        raise HTTPException(
            status_code=404, detail={"error": "Task not found"}
        )

    conn.close()
    return {"id": task_id, "title": task.title, "done": task.done}

@app.delete("/tasks/{task_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_task(task_id: int):
    """Removes a task by ID from the database."""
    conn = get_db_connection()
    cursor = conn.cursor()

    cursor.execute("DELETE FROM tasks WHERE id = ?;", (task_id,))
    conn.commit()

    if cursor.rowcount == 0:

```

```
conn.close()
raise HTTPException(
    status_code=404, detail={"error": "Task not found"}
)
```

```
conn.close()
return None
```